REINFORCEMENT LEARNING

LABORATORY REPORT — LAB 8

Function Approximation on the 1000-State Random Walk

Answers to the thirteen laboratory questions on gradient Monte Carlo and semi-gradient TD prediction with state aggregation, tile coding, and polynomial / Fourier bases, with executed experiments reproducing Figures 9.1–9.10 of the reference text.

Name	Amartya Ranjan
Registration Number	23BRS1309
Date of Submission	17 September 2026

Based on Sutton & Barto, *Reinforcement Learning: An Introduction*, 2nd ed. — Chapter 9

Contents

1 Objective, Environment and Program Overview 1

2 Question 1 - Why is function approximation needed? 2

3 Question 2 - How does one parameter learn about many states? 3

4 Question 3 - What exactly is being learned by Gradient Monte Carlo? 4

5 Question 4 - Calculating an actual parameter update 5

6 Question 5 - Why delta is not the change to the value function 6

7 Question 6 - The role of grad_w V(s; w) 7

8 Question 7 - Why the TD algorithm is "semi-gradient" 8

9 Question 8 - What happens when n changes? 8

10 Question 9 - Why is the "true value" calculated before learning? 10

11 Question 10 - Why state aggregation is piecewise-constant 11

12 Question 11 - Why tile coding generalizes more smoothly 13

13 Question 12 - Same algorithm, different representations, different curves 14

14 Question 13 - Separating the three sources of learning behaviour 16

15 Conclusion 17

1 Objective, Environment and Program Overview

The objective of this laboratory is to study **on-policy prediction with function approximation** (Sutton & Barto, Chapter 9) on the 1000-state random walk, using the reference program `random_walk.py` as the object of analysis. The program implements gradient Monte Carlo prediction and semi-gradient n-step TD prediction under three different parameterisations of the value function: state aggregation, tile coding, and linear bases (polynomial and Fourier). The lab questionnaire asks thirteen questions about this program; this report answers each one in depth, with the relevant code identified, the reinforcement-learning concept named, and the consequence for learning explained. Every claim that depends on a numerical experiment is backed by an executed notebook (`Lab8_RandomWalk_AmartyaRanjan_23BRS1309.ipynb`) whose figures are reproduced in the answer sections below.

The environment is an episodic Markov reward process. An episode starts in the central state 500; on each step the agent moves left or right with probability 0.5 and then jumps uniformly by 1 to 100 states in that direction. Left terminal state 0 pays reward -1, right terminal state 1001 pays +1, and every non-terminal transition pays 0. With gamma = 1 the value of each state under the random policy is the probability-weighted average of the two terminal payoffs, which produces a monotonically increasing, odd-symmetric curve from about -0.92 at state 1 to +0.92 at state 1000.

Component	Setting in the program
States	1000 non-terminal states 1..1000; terminals 0 and 1001 (END_STATES)
Start state	500 (centre of the chain, START_STATE)
Policy	Random: left/right with probability 0.5 each (get_action)
Dynamics	s→clip(s + A·U{1,...,100}); stride drawn by step()
Rewards	0 on all non-terminal transitions; +1 right exit, -1 left exit
Discount	gamma = 1 (undiscounted episodic prediction)

Table 1 - Environment specification of the 1000-state random walk

The submitted notebook reorganises the original script into a question-driven walkthrough and applies a small number of engineering changes that do not alter the learning logic: the Agg backend is removed so plots render inline; the dynamic-programming reference and the per-episode RMS evaluation are vectorised with NumPy; the experiment budgets are scaled to keep the complete run under two minutes; a fixed seed makes results reproducible; and two shadowed local names are renamed for clarity. Table 2 records the correspondence between the original settings and those used here.

Experiment	Original random_walk.py	This submission
True value (DP)	asynchronous sweeps, Python loops	vectorised sweeps, same stopping rule, same fixpoint
Fig. 9.1 (Q10)	100,000 episodes, alpha = 2e-5	20,000 episodes, alpha = 1e-4 (same alpha·episodes)
Fig. 9.2 right (Q8)	100 runs × 10 values of n × 11 alphas	10 runs × 6 values of n × 11 alphas
Fig. 9.10 (Q11)	5,000 episodes × 2 representations	5,000 episodes × 2 (vectorised evaluation)
Fig. 9.5 (Q12)	5,000 episodes × 6 combinations	5,000 episodes × 6 (vectorised evaluation)

Table 2 - Experiment budget: original program vs. submitted notebook

Software environment. Python 3.12.14, NumPy 2.1.3, Matplotlib 3.9.2, executed with the notebook kernel; random seed fixed at 2026. All figures in this report were produced by the executed notebook in a single top-to-bottom run of approximately 68 seconds.

2 Question 1 - Why is function approximation needed?

> **CODE FOCUS**
> `N_STATES = 1000 · ValueFunction(10) - one parameter per 100 states · gradient_monte_carlo(distribution=...) records where learning actually happens`

A tabular learner would allocate **one independent memory per state**: 1000 floating-point entries, each updated only when that exact state is visited, each carrying its own estimation error, and each completely ignorant of every other entry. In this environment that design is mismatched to the data. Every episode starts at state 500, lasts roughly 75 to 100 steps, and therefore visits only a narrow band of the state space; the lower panel of Figure 3 shows the resulting on-policy distribution, a dome centred on 500 with tails that fall below 0.1 percent of visits near the extremes. After even 100,000 episodes the tails of the table would remain poorly sampled, and the 1000 estimation problems would be solved to very different levels of accuracy. A tabular method also learns nothing transferable: an update at state 520 leaves the estimate at 521 untouched, although the two states are nearly interchangeable in value.

Function approximation replaces the 1000-slot table with a parameter vector of dimension d much smaller than 1000 - here d = 10 for state aggregation, d = 300 for tile coding, and d = 21 for the order-20 bases - and defines the estimate as a function V(s; w) computed from those shared parameters. The estimation problem changes character: instead of 1000 independent averages, the agent fits a low-dimensional surface to all of its experience at once. Every update modifies the surface globally or regionally, so information flows between states automatically.

The advantage obtained by **parameter sharing** is therefore generalization in the statistical sense: experience gathered at one state improves the predictions of many states that were never or rarely visited, because they share the parameters activated by the visited one. This has three practical consequences that the lab experiments make visible. First, sample efficiency: 20,000 episodes suffice to fit a 10-group approximation to RMS error 0.057 (Figure 3), whereas a table with 1000 free parameters would remain noisy in its unsampled regions. Second, memory and computation shrink from O(1000) to O(d). Third, the method extends beyond enumerable state spaces - the same code would work for continuous states once the features are defined, which a table cannot do.

1000 → 10

independent values replaced by 10 shared parameters (100 states per parameter) - generalization is bought with a small amount of representation bias

3 Question 2 - How does one parameter learn about many states?

> **CODE FOCUS**
> `ValueFunction.value/update: group_index = (state - 1) // self.group_size, group_size = N_STATES // num_of_groups`

The class implements a hard partition of the state space into consecutive blocks, and every answer below follows from the integer division in that mapping. With 10 groups the group size is group_size = 1000 // 10 = **100 states**, so group k covers the states 100k+1 through 100k+100.

Question	Answer inferred from the code
1. States per group	1000 // 10 = 100 states
2. Parameter used by state 137	(137 - 1) // 100 = 1, i.e. w[1], the second group parameter
3. States affected by changing w[1]	every state with the same index: 101 through 200, all 100 of them
4. Why called generalization	one update, triggered by a visit to a single state, instantly corrects the estimates of 99 states that were never visited in that episode

Table 3 - Mapping inference for state 137 under a 10-group ValueFunction

The notebook verifies the third answer empirically: starting from all-zero parameters, a single call to update(0.5, 137) changes exactly 100 state estimates, and the changed set is precisely the states 101..200 reported by the vectorised evaluator. This is the mechanism the question is pointing at - the parameter is

not attached to a state, it is attached to a **group**, and the state merely selects which parameter its group exposes.

It is called *generalization* because statistical strength is transferred from experienced states to inexperienced ones. Under the on-policy distribution the middle groups receive thousands of updates while the extreme groups receive few; sharing lets the rarely visited extremes inherit a sensible value (the average of the returns that do reach them) rather than staying at their initial guess. The price is **banding bias**: states inside a block are represented identically, so the approximation cannot express any variation within the 100-state band, whatever the true value function does there. Question 10 examines exactly this trade-off in picture form.

4 Question 3 - What exactly is being learned by Gradient Monte Carlo?

> **CODE FOCUS**
> `gradient_monte_carlo(): the interaction loop stores the final reward; the update loop reuses the same scalar for every visited state: delta = alpha*(reward - V)`

The return from time t is defined as the discounted sum of future rewards. Expanding it in this environment separates the two structural facts the question asks about. All rewards are zero except the terminal one, so every intermediate term vanishes, and only the terminal payoff survives, discounted by the number of steps between t and termination:

$$G_t = R_{t+1} + \gamma R_{t+2} + \ldots + \gamma^{T-t-1} R_T = 0 + 0 + \ldots + 0 + \gamma^{T-t-1} \cdot R_T$$

With **gamma = 1** the discount factor equals one, so the surviving term is simply R_T regardless of t: an episode that ends at the right terminal state has $G_t = +1$ for **every** state visited on that episode, and one that ends left has $G_t = -1$ throughout. The whole trajectory shares a single target. This is why the program can compute the reward once inside the interaction loop and reuse the same scalar inside the update loop - a legitimate simplification, not an approximation.

The notebook checks this on a sampled episode: computing the returns backwards as G = r + 1.0·G for every time step reproduces the final reward at every t (64-step episode ending at state 0 with reward -1; all 64 returns equal -1). What is being learned, then, is the **probability-weighted average of the two terminal payoffs**: gradient MC drives V(s; w) toward the fraction of episodes from s that exit right minus the fraction that exit left, which is exactly $v_\pi(s)$.

If intermediate transitions produced non-zero rewards this would no longer be true. G_t would sum a different window of rewards for each t - earlier states see strictly more reward terms than later ones - so the target would be trajectory-dependent and per-time-step. The program would need to accumulate returns backwards along the trajectory, $G_t = R_{t+1} + \gamma G_{t+1}$, and with $\gamma < 1$ even the shared terminal payoff would be weighted differently for each state (γ^{T-t-1}). The single-scalar shortcut exists precisely because

both facts - zero intermediate rewards and unit discounting - hold simultaneously.

5 Question 4 - Calculating an actual parameter update

> **CODE FOCUS**
> `linear form: V(s,w) = w·x(s), so grad_w V(s,w) = x(s) = [1, 0.5]; delta = alpha*(G - V); w += delta * x(s)`

For a linear approximator the gradient of the prediction with respect to the weights is the feature vector itself, so the gradient Monte Carlo update can be evaluated by pure arithmetic. The notebook reproduces every line of Table 4 numerically. Taking w = [0.4, 0] (any weight vector with w·x(s) = 0.4 represents the same prediction) gives:

Step (role)	Expression	Value
1. Prediction	V(s; w) = w·x(s)	0.4
2. Target	G	1
3. Error	G - V(s; w)	1 - 0.4 = 0.6
4. Scaled error (delta)	alpha·(G - V)	0.1 × 0.6 = 0.06
5. Gradient	grad_w V(s; w) = x(s)	[1, 0.5]
6. Parameter change	delta·x(s)	[0.06, 0.03]
7. New weights	w + delta·x(s), from w = [0.4, 0]	[0.46, 0.03]
8. New prediction	w_{new}·x(s)	0.46 + 0.03×0.5 = 0.475

Table 4 - Step-by-step gradient Monte Carlo update for Q4

The prediction moves from 0.4 toward the target 1 by $0.06 \times ||x(s)||^2 = 0.075$, reaching 0.475 - a partial, cautious step rather than a full correction. That is characteristic of stochastic gradient descent with a small step size: no single visit is allowed to drive the parameters to fit one sample exactly, because the same parameters must serve every other state and every future sample.

Why both parameters change although only one state was visited. The update is addressed to the *parameter vector*, not to the state. Both components of x(s) are non-zero - the constant feature with weight 1 and the half-valued feature with weight 0.5 - so both weights participate in representing V(s) and both receive a share of the error proportional to their feature value: w_1 moves by 0.06 and w_2 by 0.03. This is generalization operating at the finest level: any other state whose feature vector loads the same components is carried along by the same parameter change, which is precisely how one visit improves predictions at many states. If a feature were zero at s, its weight would receive no gradient from this visit - the feature vector decides where the learning lands.

6 Question 5 - Why delta is not the change to the value function

> **CODE FOCUS**
> `prediction -> reward - prediction -> delta -> value_function.update(...); each representation implements update() differently`

The quantity delta = alpha·(target - prediction) lives in **prediction space**: it is how much the *estimate at the visited state* should move. A tabular program can apply it directly, V[s] += delta, only because for a table the state's entry is itself the parameter - the gradient is a one-hot vector. With function approximation the parameters are shared, so the scalar must first be **translated into parameter space** by the gradient before it can be applied, and each representation performs that translation differently inside its own update() method.

Representation	Parameter update implemented	Effect
State aggregation	params[group(s)] += delta - the entire delta goes to one weight	1 parameter moves by 0.06; the estimates of 100 states shift by 0.06
Tile coding	delta /= 50, then each of the 50 active tiles gets += delta/50	50 parameters move by 0.0012 each; V(137) still moves by exactly 0.06
Polynomial / Fourier	weights += delta · x(s), distributing the delta proportionally to the feature values	order+1 parameters move by different amounts; V(137) moves by delta·$\|\|x\|\|^2$, not delta

Table 5 - What update(delta, s) does in each representation (delta = 0.06 at state 137)

The notebook demonstrates the contrast with a single shared delta of 0.06: aggregation moves one weight and 100 state estimates; tile coding moves 50 tiles by delta/50 = 0.0012 each, leaving the visited state's estimate changed by exactly delta while its neighbours change fractionally; the polynomial update multiplies the delta by the feature vector, so the visited state's estimate moves by delta·$\|\|x\|\|^2$ - a different amount from delta altogether.

This is why the program cannot use one ordinary table-update routine for all three representations. If update() blindly added delta to every active parameter, tile coding would move the visited state's estimate by 50×delta (fifty times too far) and the bases would move it by delta·$\Sigma_i x_i(s)$, both wrong. The gradient factor in the update rule is the bookkeeping that keeps the effective prediction change consistent while the parameter-space consequences differ - and the differences are exactly what gives each representation its own generalization behaviour (Questions 10-12).

7 Question 6 - The role of grad_w V(s; w)

CODE FOCUS

```
general semi-gradient rule: w += alpha · e · grad_w V(s,w), e = scaled error;
implemented inside the three update() methods
```

The learning rule is representation-agnostic because all representation-specific knowledge is packed into the gradient vector. The gradient answers one question per parameter: *if this weight changes by one unit, how much does the estimate at s change?* Because all three approximators are linear in their parameters, $V(s; w) = w \cdot x(s)$, the gradient is simply the feature vector x(s) - but the three feature vectors are structurally different objects, and that difference is the whole story.

Representation	grad_w V(s; w) in this program	Interpretation
State aggregation	one-hot vector: 1 for the state's group, 0 elsewhere	a selector: picks the one parameter responsible for s
Tile coding	1 in the 50 active tiles (equivalently 1/50 each after the equal split in update())	a shared receptive field: the 50 tiles overlapped by s
Polynomial basis	$x(s) = [1, s, s^2, ..., s^{order}]$ with s normalised to [0,1]	dense and correlated: every weight participates, unevenly in s
Fourier basis	$x(s) = [1, \cos(\pi s), \cos(2\pi s), ..., \cos(order \cdot \pi s)]$	dense and orthogonal on [0,1]: every weight participates, evenly

Table 6 - The gradient of V(s; w) for each representation

The same rule works with three representations because it is just **stochastic gradient descent on the squared error** $(G - V(s; w))^2$: whatever differentiable form V has, the negative gradient of the squared error with respect to w is proportional to e·grad_w V, and following it reduces the error on the current sample. The gradient differs across representations because it is the *derivative of the chosen parameterisation* - changing what V is made of changes how sensitive the output is to each weight. The notebook computes the four gradient vectors at state 137 and shows the contrast concretely: the aggregation gradient is a single spike, the tile-coding gradient is 50 spikes of height 1/50, the polynomial gradient rises with the powers of 0.137, and the Fourier gradient is a smooth set of cosines.

Two consequences follow. First, the gradient is a *unit of generalization*: parameters with large gradient components receive more of every error, so the shape of x(s) decides which regions of the state space move together. Second, the norm $||x(s)||^2$ acts like a state-dependent effective step size, which is why the same alpha behaves differently across representations - a point developed in Question 12.

8 Question 7 - Why the TD algorithm is "semi-gradient"

> **CODE FOCUS**
> `semi_gradient_temporal_difference() with n = 1: target = rewards[t+1] + value(states[t+1]); update uses only grad of V(states[t], w)`

For n = 1 the update target assembled by the program is the one-step bootstrapped return $R_{t+1} + \gamma V(S_{t+1}; w)$. The comparison with the Monte Carlo target is a comparison between data and model: G_t is an *observed* quantity, completely determined by the episode and independent of the parameters, whereas the TD target contains **a guess produced by the very function being learned**. That bootstrap term is the part of the target that depends on the current w.

A fully gradient-based treatment of the TD error would differentiate the residual $(R_{t+1} + \gamma V(S_{t+1}; w) - V(S_t; w))$ through *both* occurrences of w, propagating gradient through the target as well as through the prediction. The program - like the standard semi-gradient TD algorithm of Chapter 9 - deliberately does not: it treats the target as a constant and uses only $\text{grad}_w V(S_t; w)$, the gradient of the *current prediction at the state being updated*. Hence the name **semi**-gradient: half of the residual is differentiated, the other half is frozen.

Why freeze it? Three reasons. **Stability:** differentiating through a target that itself moves at every update can drive the bootstrapping term to compensate for its own changes - the classical source of divergence in naive TD variants. **Simplicity and cost:** the update needs only one gradient evaluation, of the same form as the MC update. **Theory:** for linear approximation, semi-gradient TD converges robustly to the fixed point of the projected Bellman equation, the best approximation of v within the representable set - it does not need the true gradient of any single objective to be useful. The price is **bias**: the target inherits whatever error $V(S_{t+1}; w)$ currently has, so learning propagates estimates rather than only data. The notebook makes the w-dependence visible by nudging one parameter and watching the TD target move while the MC target stays fixed.

9 Question 8 - What happens when n changes?

> **CODE FOCUS**
> `n-step target: sum of rewards[t+1..t+n] + bootstrap V(states[t+n], w) if t+n <= T; experiment = scaled Figure 9.2 (right): RMS vs alpha for n in {1,...,32}`

The n-step target mixes n real rewards with a single bootstrap:

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \ldots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}; w)$$

As n grows, fewer bootstrapped estimates and more actually observed rewards enter the target; at $n \geq T - t$ the bootstrap disappears entirely and $G_{t:t+n}$ is the full Monte Carlo return. Increasing n therefore makes the algorithm **more like Monte Carlo, not more like one-step TD**. The statistical character of the target

changes accordingly: bootstrapping removes variance by substituting a current estimate for the random remainder of the episode, at the cost of bias toward whatever the estimate already believes; longer n removes that bias but reinstates the variance of long, random reward sequences. In this environment the reward is a single ±1 terminal event, so an n-step target is (almost) a Bernoulli sample until the bootstrap drops out - the variance increase with n is stark and visible.

Figure 1 shows the scaled reproduction of Figure 9.2 (right): 10 independent runs, 10 episodes each, for n in {1, 2, 4, 8, 16, 32} and alpha from 0 to 1. The best combination found is n = 8 with alpha = 0.3 at RMS 0.284, echoing the book's finding that intermediate n performs best on this problem. The figure also shows the second consequence of growing n: the usable learning-rate range shrinks - n = 32 tolerates only small alpha before its high-variance targets destabilise the weights, while n = 1 remains stable even at alpha = 1 but plateaus at a higher error because its biased, heavily bootstrapped target limits how far the weights can move within 10 episodes.

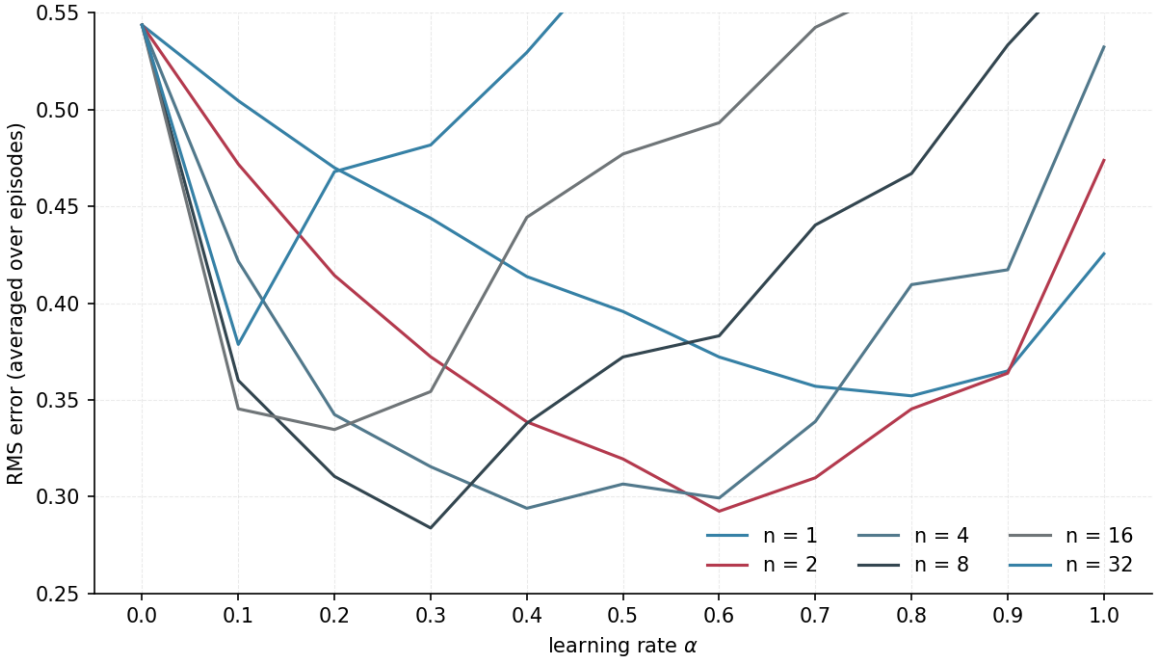


Figure 1 - Semi-gradient n-step TD on the 1000-state random walk: RMS error vs learning rate for several n (10 runs × 10 episodes, 20-group aggregation). Intermediate n = 8 achieves the lowest error.

10 Question 9 - Why is the "true value" calculated before learning?

> **CODE FOCUS**
> `true_value = compute_true_value() - model-based iterative policy evaluation (dynamic programming), run before any learning experiment`

How the reference is obtained. compute_true_value() performs exact iterative policy evaluation of the random policy - a form of asynchronous value iteration. It initialises the value array with the linear guess $v_0(s) = (2s - 1001)/1001$, whose terminal entries already equal the two payoffs, and then sweeps over all 1000 non-terminal states, replacing each value by the exact Bellman expectation of the random policy over all 200 possible (action, stride) combinations: V(s) gets $(1/200)\cdot\Sigma$ V(clip(s + a·u)). Because the update for state s is applied immediately within the sweep (asynchronous, Gauss-Seidel style), the iteration converges quickly, and the loop stops when the total absolute change drops below 0.01. This is pure dynamic programming: the transition model is known and fully enumerated, no sampling is involved, and no learning algorithm runs.

Why it is useful although the algorithms estimate the value themselves. The learning algorithms are model-free: they see only sampled episodes, and their output depends on sample noise, step size, number of episodes, and representation capacity. The DP solution is the exact yardstick v_π that says what all of this effort is trying to approximate. Without it, the experiments could only compare learners against each other - one curve lower than another - but never quantify *how far from the truth* any of them is. It also plays a calibration role for the program itself: the initialisation comment calls the linear guess "a promising guess", and the DP sweep is what turns that guess into the certified reference.

Why the RMS plots need it. Every evaluation figure in this lab computes RMS = $\text{sqrt}((1/1000)\cdot\Sigma_s(V(s; w) - v_\pi(s))^2)$, which requires the reference value at **all 1000 states simultaneously** - including the states the agent almost never visits (the tails of the distribution in Figure 3). Sampled experience cannot supply those; no amount of episodes starting at 500 produces a reliable return average for state 5. Only the model-based DP sweep can. Figure 2 shows the sweep-by-sweep convergence of the reference: the linear guess is already close, and the sweeps sharpen it into the exact curve (the vectorised implementation converges to the same stopping criterion in 85 sweeps).

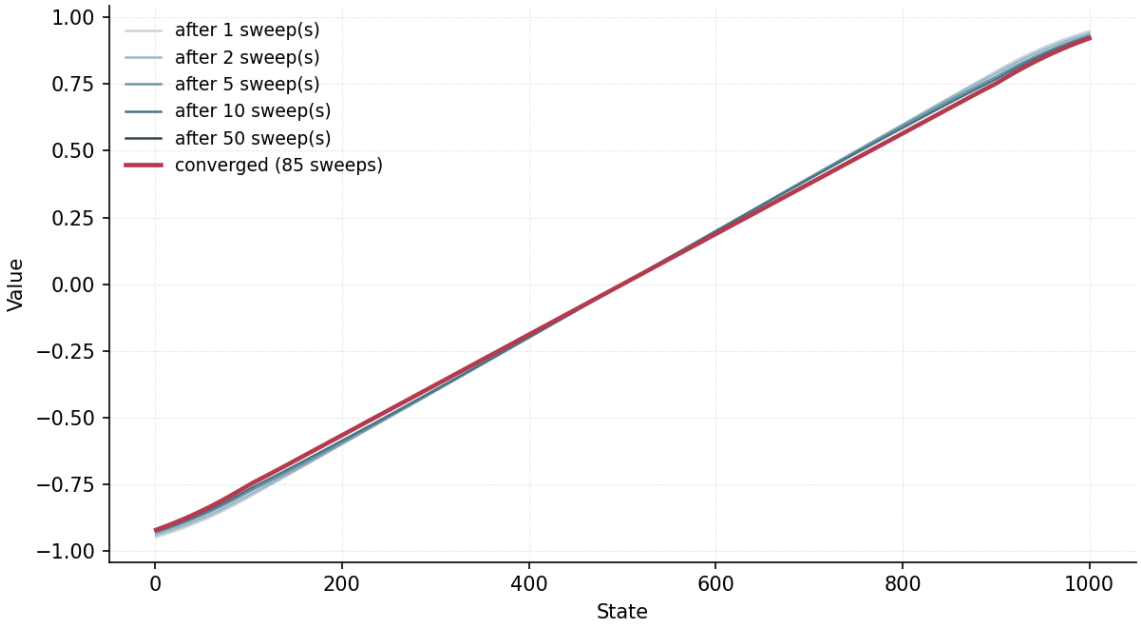


Figure 2 - Convergence of the DP reference: value function after 1, 2, 5, 10 and 50 sweeps vs. the converged reference (85 synchronous sweeps, sum of absolute change below 0.01).

11 Question 10 - Why state aggregation is piecewise-constant

CODE FOCUS

```
V(s) = params[(s-1) // 100] with 10 groups - the top panel of Figure 3 is the required sketch; the bottom panel shows the on-policy state distribution
```

With 10 groups the approximation is $V(s) = w_{(s-1)//100}$: the estimate is one constant per 100-state block, so the learned "curve" is a staircase with ten steps, drawn in the top panel of Figure 3 against the DP reference. The graph has **abrupt jumps at the group boundaries** because nothing in the representation enforces continuity: state 100 reads w_0 and state 101 reads w_1, two parameters that are updated independently by whichever returns happen to reach their groups. Inside a block the true value function keeps rising, but the approximation cannot express it - the whole block shares one flat number, typically that group's visit-weighted average of the terminal rewards.

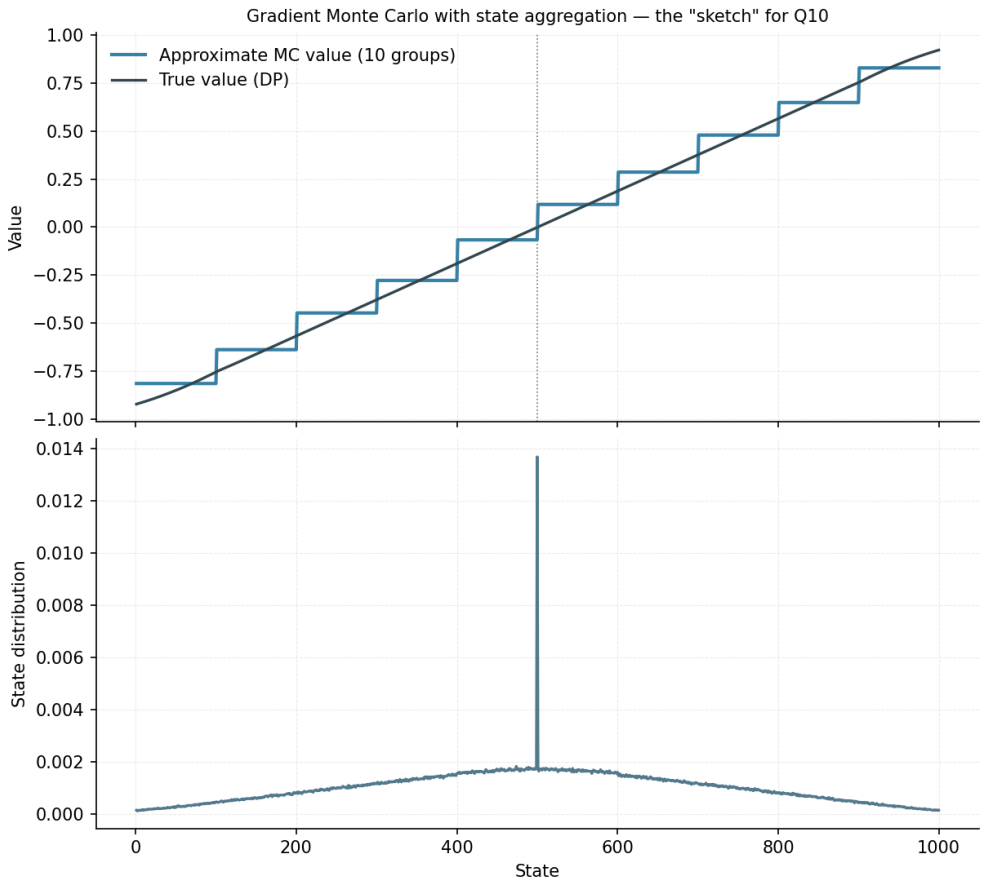


Figure 3 - Gradient Monte Carlo with 10-group state aggregation (20,000 episodes, alpha = 1e-4; final RMS 0.057). Top: the piecewise-constant approximation vs. the true value - the sketch requested by Q10. Bottom: the on-policy state distribution accumulated during learning, with its spike at the start state 500.

Would more groups necessarily be better? No - and the reason separates **representation capacity** from **learning quality**. Raising the number of groups reduces the bias floor: a 20-group staircase can hug the true curve more closely than a 10-group one, and in the limit of 1000 groups the representation recovers the full tabular function. But capacity is only an upper bound on quality. Each group's parameter is estimated from the episodes that reach it, so doubling the groups halves the data per parameter and raises the variance of every estimate; with a fixed episode budget the finer partition can end up *noisier* than the coarse one, and the extreme groups - already starved by the on-policy distribution - starve first. At the far extreme, 1000 groups is the tabular method with no generalization at all, the worst configuration for this sample budget (Question 1). The practical message is that the number of groups must be tuned jointly with the data budget: the book's own experiments use 10-20 groups for exactly this reason.

12 Question 11 - Why tile coding generalizes more smoothly

CODE FOCUS

```
TilingsValueFunction(50, 200, 4): 50 tilings, tile width 200, tilings offset by 4
states; each state activates one tile per tiling - 50 parameters
```

In tile coding a state's estimate is the **sum of 50 tile weights**, one active tile per tiling. The tilings are shifted from one another by 4 states (tilingOffset = 4), so the boundaries of the 50 tilings form a fine comb with one edge every 4 states. Move the state by one step: in the great majority of tilings the **same tile is still active** - only the tilings whose boundary lies between the two states switch tiles - so 47 to 49 of the 50 active parameters are shared with the previous state. The estimate therefore changes by a small fraction of a tile-weight difference instead of jumping wholesale: neighbouring states inherit most of each other's value automatically.

Property	State aggregation	Tile coding
Active parameters per state	1	50 (one per tiling)
Resolution of a state change	100-state blocks	4-state steps (the tiling offset)
Value at neighbouring states	identical inside a block	differs only where a tile boundary is crossed
Smoothness of V(s)	staircase, 100-wide steps	fine staircase, amplitude about 1/50 per tile switch
Parameters (this configuration)	10	50 × 6 = 300
Generalization footprint	hard group of 100 states	graded overlap of 50 receptive fields

Table 7 - State aggregation vs. tile coding in this program

Tile coding is the smoother representation, and the mechanism is **overlap plus offset**: coarse coding with many shifted receptive fields turns parameter sharing into a graded kernel, where each state change touches only a sliver of the total parameter mass. State aggregation is the degenerate case - a single, non-overlapping tiling - so its generalization is all-or-nothing. Figure 4 shows the consequence for learning: under the identical gradient-MC algorithm and the same decaying step size, the 5-group aggregation plateaus around 0.15-0.20 RMS (its representation floor plus residual noise), while tile coding keeps descending to roughly 0.05-0.07 - several times lower - because its 300 parameters resolve the value curve at 4-state granularity.

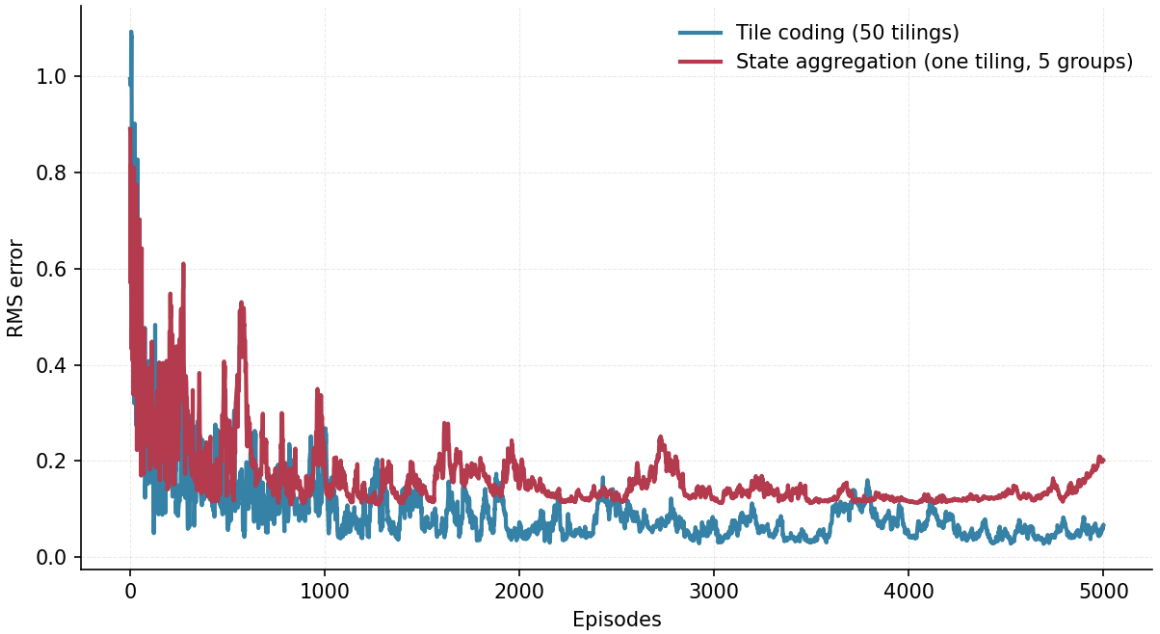


Figure 4 - Gradient Monte Carlo learning curves (5,000 episodes, decaying alpha = 1/(ep+1)): tile coding (50 tilings) vs. one-tiling state aggregation (5 groups of 200 states).

13 Question 12 - Same algorithm, different representations, different curves

CODE FOCUS

`figure_9_5(): BasesValueFunction(order, POLYNOMIAL_BASES or FOURIER_BASES) under the same gradient_monte_carlo(); alphas 1e-4 (poly) and 5e-5 (Fourier)`

The algorithm fixes the *targets* and the *timing* of updates; the features fix the *geometry* in which those updates act. Two agents with identical episodes but different basis functions optimise in differently shaped parameter spaces, so equal effort does not purchase equal accuracy. Four concrete mechanisms drive the gap.

Basis functions and approximation capacity. After normalising s into [0,1], the polynomial features 1, s, s^2, ..., s^{20} are strongly correlated - all monotone, nearly collinear, and almost flat over most of the range with sharp changes near s = 1. Low orders simply cannot span the S-shaped, odd-symmetric true value (their best fit leaves a large residual), while the extra capacity of high orders concentrates where the data is scarce. Fourier features $\cos(i \cdot \pi s)$ are **orthogonal on [0,1]**: each order adds an independent direction, the odd symmetry about s = 0.5 is expressible with a handful of low-frequency cosines, and the representable set matches the target's shape far better at the same parameter count.

Parameter sharing and generalization. A polynomial weight w_i scales the global trend s^i; an update at a large-s state (where s^i is large) moves estimates near s = 1 strongly while barely touching the low end - the correction is spatially unbalanced. A Fourier weight spreads its correction over a spatially distributed

cosine lobe, giving more even generalization across the chain. **Sensitivity to the learning rate.** The effective step at s is alpha·e·$||x(s)||^2$; for polynomials this quantity varies wildly with s and grows with the order, so one alpha cannot suit all states - the program must use 1e-4, and convergence is jittery. Orthogonalised Fourier features keep $||x(s)||^2$ comparable across s and orders, so even the shared small alpha produces steady descent.

Figure 5 shows the outcome with everything else held fixed (same gradient MC, same episodes, same rewards): Fourier finishes at 0.081 / 0.069 / 0.051 RMS for orders 5 / 10 / 20, while polynomial stalls at 0.192 / 0.144 / 0.206 - and for polynomials, *adding order makes the final error worse*, the signature of ill-conditioned features rather than insufficient capacity. **Is the learning algorithm alone sufficient to determine performance?** Clearly not: performance is a property of the (algorithm × representation) pair. The algorithm decides what information is available and when; the representation decides what can be expressed with it and how far each update travels.

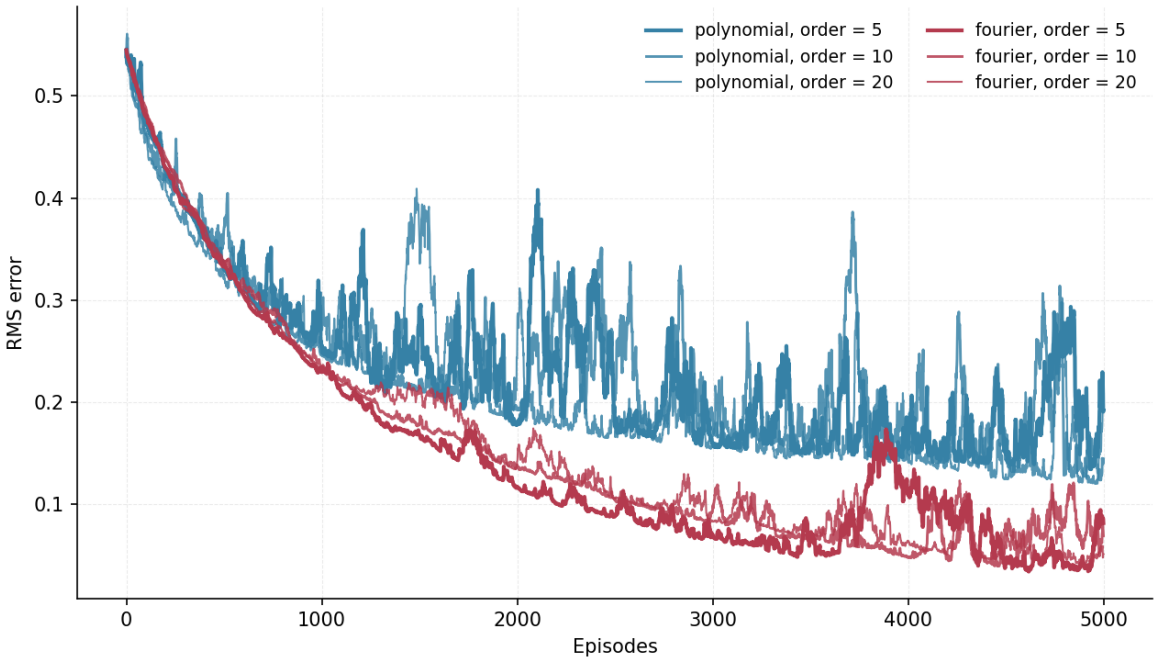


Figure 5 - Gradient Monte Carlo with polynomial vs. Fourier bases (5,000 episodes, single run as in the original program): RMS error over episodes for orders 5, 10, 20.

14 Question 13 - Separating the three sources of learning behaviour

> **CODE FOCUS**
> `gradient_monte_carlo / semi_gradient_temporal_difference (target) · delta = alpha*(target - prediction) (update) · the three value-function classes (generalization)`

The program's learning behaviour is produced by three separable components, and the questionnaire's menu maps onto them one-to-one.

Behaviour	Responsible component	Where it lives in the code
Learning target	The algorithm: Gradient MC or semi-gradient n-step TD	gradient_monte_carlo(): the terminal reward as G for every visited state; semi_gradient_temporal_difference(): n collected rewards plus the bootstrapped V(states[t+n], w)
Parameter update	The update rule and the learning rate alpha	delta = alpha·(target - prediction); each update() applies the scaled error; alpha is fixed (2e-4), tuned per experiment, or decayed as 1/(ep+1)
Generalization across states	The value-function representation	ValueFunction (one-hot group features), TilingsValueFunction (50 overlapping tiles), BasesValueFunction (dense polynomial/Fourier features)

Table 8 - Component responsibility map

Why changing only the representation changes the final approximation even when the algorithm is unchanged. Given the same targets and the same alpha, the direction and spread of every update in parameter space is still the feature vector x(s) (Question 6). Changing the representation changes x(s), and with it three things at once. It changes *which* parameters move and how many (one group weight vs. 50 tiles vs. 21 basis weights), hence which regions of the state space learn together. It changes the *conditioning* of the optimisation - correlated polynomial features fight each other while orthogonal Fourier features cooperate (Question 12). And it changes the *best achievable error* for the given parameter count, the bias floor that no amount of extra episodes can beat (Questions 10 and 11). The experiments demonstrate the separation cleanly: Q8 varied the target alone (n) with a fixed representation and got different curves; Q10/Q11 varied the representation alone under the same MC algorithm and got different asymptotes; Q12 varied only the features with the same algorithm and alpha and got dramatically different learning speeds.

The engineering lesson mirrors the theory: when a learner underperforms, the three levers are distinct diagnostics. If the target is the problem, adjust n or move between MC and TD; if updates are the problem, adjust alpha or its schedule; if generalization is the problem, redesign the features. Changing

one lever at a time - exactly as this program permits by swapping the value-function class while keeping gradient_monte_carlo() untouched - is what makes the diagnosis possible.

15 Conclusion

This laboratory used the 1000-state random walk to take Chapter 9 apart component by component. Function approximation was shown to be a necessity, not a convenience: with a start-state-centric visit distribution, 1000 independent memories cannot be filled, while 10-300 shared parameters fit the value curve to a few hundredths of RMS error. The mechanics of sharing were traced precisely - the integer group mapping that makes one parameter answer for 100 states, the scalar delta that must be translated by a gradient before it can touch parameters, and the feature vector that decides where each error lands. The simplification of gradient MC under gamma = 1 with a single terminal reward was derived and verified on sampled episodes, and the semi-gradient label was justified by exhibiting the w-dependence of the TD target that the algorithm deliberately leaves undifferentiated.

The experiments then isolated the three levers of learning behaviour. Growing n moves the n-step target from biased bootstrapping toward high-variance Monte Carlo, with n = 8 the sweet spot on this task. Representation quality appeared in two distinct forms: sharpness - tile coding's 50 offset tilings resolve the value function at 4-state granularity and descend far below the aggregation plateau - and conditioning - Fourier bases turn the same algorithm and learning rate into smooth convergence while correlated polynomial bases stall regardless of order. Throughout, the DP-computed reference value function was what made honest measurement possible.

The submitted notebook preserves the learning logic of random_walk.py while making it faster (vectorised DP reference and RMS evaluation), reproducible (fixed seed), and pedagogically structured (one section per question, each with a runnable demonstration). All thirteen answers above are backed by executed code and figures produced from that notebook in a single run.